

Sum.by.strata.redone_documentation

Fred Cudhea

14 July, 2025

You'll read in this script in "Complete.PIF.Analysis.partTwo.cluster.r". It defines a bunch of function used for summarizing the output (and a couple obsolete functions that have seen been replaced. Since this code is just defining functions, there is no harm in not commenting them out).

This function takes in allmort (simulation of attributable mortality/incidence) and total mort (simulation of total mortality/incidence) to calculate "reverse-engineered" PIFs, which is simply the attributable mort/incidence divided by total mort/incidence. What's the point when we calculate the PIFs directly in part one? We want to be able to report PIFs for broader subgroups (not just by age/sex/race) and this is the way to do that. First step in the function is two merge totalmort and allmort (to ensure everything matches up by subgroup), next is to identify column indices for the totalmort and allmort simulations, and then finally use those indices to take allmort sims and divide by total mort sims. **Note that this function only works when your subgroups are age/sex/race specific.** Output is sims for reverse-engineered PAFs for all subgroups.

```
#only works on specific allmort file created by my code
##use covar="overall" for all

##function of calculating RE PAFs sum stats
get.RE.PAFs.draws<-function(allmort, totalmort) ##for now, only works for age/sex/race
{
  merged<-merge(allmort, totalmort)

  allmort.starting.point<-which(names(merged)== "V1")
  allmort.ending.point<-which(names(merged)== "riskfactor")-1
  totalmort.starting.point<-which(names(merged)== "X1")
  totalmort.ending.point<-dim(merged)[2]

  RE.PAFs.info<-merged[,which(names(merged) %in% c("age", "female", "race", "outcome", "mean..food

  RE.PAFs<-cbind(merged[,which(names(merged) %in% c("age", "female", "race", "outcome", "mean..fo
    merged[,allmort.starting.point:allmort.ending.point])/merged[,totalmort.starting.p
  RE.PAFs<-RE.PAFs[order(RE.PAFs$riskfactor, RE.PAFs$outcome, RE.PAFs$female, RE.PAFs$age, RE.PAFs

  return(RE.PAFs)
}
```

Literally the same thing as above, but with the added step of getting summary stats from the simulated RE PIFs.

```

get.RE.PAFs<-function(allmort, totalmort) ##for now, only works for age/sex/race
{
  merged<-merge(allmort, totalmort)

  allmort.starting.point<-which(names(merged)== "V1")
  allmort.ending.point<-which(names(merged)== "riskfactor")-1
  totalmort.starting.point<-which(names(merged)== "X1")
  totalmort.ending.point<-dim(merged)[2]

  RE.PAFs.info<-merged[,which(names(merged) %in% c("age", "female", "race", "outcome", "mean..food

  RE.PAFs<-cbind(merged[,which(names(merged) %in% c("age", "female", "race", "outcome", "mean..fo
    merged[,allmort.starting.point:allmort.ending.point])/merged[,totalmort.starting.poi
  RE.PAFs<-RE.PAFs[order(RE.PAFs$riskfactor, RE.PAFs$outcome, RE.PAFs$female, RE.PAFs$age, RE.PAFs

  RE.PAFs.sum.stats<-sum.stats.maker.RE.PAFs(RE.PAFs, n.covar)
  return(RE.PAFs.sum.stats)
}

```

Next is a function we'll use often. It'll read in a simulation file (like attributable mortality sims) and output summary stats (mean, standard deviation, median, 2.5th and 97.5th percentile).

```

#function to get summary stats#####
sum.stats.maker<-function(allmort)
{
  allmort.dt<-as.data.table(allmort)
  vars<-paste("V", 1:n.sims, sep="")

  mort.summary <- allmort.dt[, ":(means = rowMeans(.SD, na.rm = TRUE),
                                sd.devs = apply(.SD, 1, sd),
                                medians = apply(.SD, 1, median),
                                LB = apply(.SD, 1, function(x) quantile(x, .025, na.rm = T)),
                                UB = apply(.SD, 1, function(x) quantile(x, .975, na.rm = T))
                                ),
    .SDcols = vars
  ]

  mort.summary <- mort.summary[, (vars) := NULL]

  return(mort.summary)
}

```

Next, a similar function that is used for Reverse engineered PAF draws. In addition for getting the five summary stats mentioned above, it also outputs median, 2.5th percentile, and 97.5th percentile as a percent (as opposed to as a proportion) for convenience.

```

sum.stats maker.RE.PAFs<-function(RE.PAFs)
{
  RE.PAFs.dt<-as.data.table(RE.PAFs)
  vars<-paste("V", 1:n.sims, sep="")

  RE.PAFs.summary <- RE.PAFs.dt[, "!="(RE.PAF.means = rowMeans(.SD, na.rm = TRUE),
                                     RE.PAF.sd.devs = apply(.SD, 1, sd),
                                     RE.PAF.medians = apply(.SD, 1, median),
                                     RE.PAF.LB = apply(.SD, 1, function(x) quantile(x, .025, na.
                                     RE.PAF.UB = apply(.SD, 1, function(x) quantile(x, .975, na.
                                     RE.PAF.medians.as.percent = apply(.SD, 1, median)*100,
                                     RE.PAF.LB.as.percent = apply(.SD, 1, function(x) quantile(x
                                     RE.PAF.UB.as.percent = apply(.SD, 1, function(x) quantile(x

  ),
  .SDcols = vars
]

RE.PAFs.summary <- RE.PAFs.summary[, (vars) := NULL]

return(RE.PAFs.summary)
}

```

Next is Sum.by.strata. Very important function that produces output by strata of interest (e.g, by age instead of by age/sex/race, etc...). Inputs are simulation output for attributable mort (allmort), total mort (totalmort), total population (pop), and finally, a string vector of the strata of interest (covar). This function is a bit more steps than function above so I'll break it down chunk by chunk, by the general idea is simple: Step 1 is to sum over strata of interest, Step 2 is calculate summary stats for those strata of interest.

```

Sum.by.strata<-function(allmort, totalmort, pop, covar)
{

```

First, convert input data frames to data tables and get string of variable names for the simulations for each file. (not we also rename are string vector of strata to "strata")

```

allmort.dt<-as.data.table(allmort)
totalmort.dt<-as.data.table(totalmort)
pop.dt<-as.data.table(pop)

strata<-covar

cols.to.sum.allmort<-paste("V", 1:n.sims, sep="")
cols.to.sum.totalmort<-paste("X", 1:n.sims, sep="")
cols.to.sum.pop<-paste("Y", 1:n.sims, sep="")

```

Following code sums the simulations over strata of interest. ".SDcols: are your variables of interest (in our case, the simulated values: V1, V2, etc...)."lapply(.SD, sum)" is the function you're applying: in our case summing values in each column. Whatever you specify in "by=" is the grouping of interest. Let's say it's age, as an example. For attributable mort/incidence, you're summing by age and risk factor, because you don't want to sum results for different foods together. For total mortality, you only have data by age/sex/race and outcome/disease_type so

whatever you listed in “covar”, you are limited to those five groupings (that’s what `c(strata[strata %in% c(“age”, “female”, “race”, “outcome”, “disease_type”)])` does). Total population numbers aren’t divisible by outcome/disease type, so you’re strata are limited to some combination of age/sex/race.

```
strata.sims.allmort<-allmort.dt[,lapply(.SD, sum), by=c(strata, "riskfactor"), .SDcols=cols.to]
strata.sims.totalmort<-totalmort.dt[,lapply(.SD, sum), by=c(strata[strata %in% c("age", "female", "race"),
  .SDcols=cols.to.sum.totalmort]
strata.sims.pop<-pop.dt[,lapply(.SD, sum), by=c(strata[strata %in% c("age", "female", "race")]
```

Next, we merge three output data tables together. How to merge them correctly depends on the what strata you are interested, hence the four if statements. Everything in the if statement is doing the same thing (merging the three outputs together), but they need to be implemented differently depending on whether one or more of the data tables are **one dimensional or not** (merge function basically doesn’t work if one of the things to be merged is one dimensional). So, the first if statement, where none of the three are one dimensional, then we can use the merge function without issue. In the other three cases, where either `strata.sims.totalmort` is one dimensional, we need to use `cbind` instead of `merge` to bind the files together. So if `strata.sims.pop` is one dimensional (if `length(strata[strata %in% c(“age”, “female”, “race”)])==0` is true) and/or if `strata.sims.allmort` is one dimensional (if `dim(strata.sims.totalmort)[1]>1`), then we need to use `cbind` to combine those to the rest of the files.

```
#merged<-reduce(list(strata.sims.allmort, strata.sims.totalmort, strata.sims.pop), merge)
if(length(strata[strata %in% c("age", "female", "race")])>0 & dim(strata.sims.totalmort)[1]>1)
  merged<-merge(merge(strata.sims.allmort, strata.sims.totalmort), strata.sims.pop, by=strata[
if(length(strata[strata %in% c("age", "female", "race")])>0 & dim(strata.sims.totalmort)[1]==1
{
  merge.a<-cbind(strata.sims.allmort, strata.sims.totalmort)
  merged<-merge(merge.a, strata.sims.pop, by=strata[strata %in% c("age", "female", "race")])
}
if(length(strata[strata %in% c("age", "female", "race")])==0 & dim(strata.sims.totalmort)[1]>1)
{
  merged<-cbind(merge(strata.sims.allmort, strata.sims.totalmort), strata.sims.pop)
}
if(length(strata[strata %in% c("age", "female", "race")])==0 & dim(strata.sims.totalmort)[1]==1)
  merged<-cbind(strata.sims.allmort, strata.sims.totalmort, strata.sims.pop)
```

Next is identifying the index for starting and ending of simulation values for each of the three simulated outputs.

```
allmort.starting.point<-which(names(merged)=="V1")
allmort.ending.point<-which(names(merged)==paste("V", nsim1, sep=""))
totalmort.starting.point<-which(names(merged)=="X1")
totalmort.ending.point<-which(names(merged)==paste("X", nsim1, sep=""))
pop.starting.point<-which(names(merged)=="Y1")
pop.ending.point<-which(names(merged)==paste("Y", nsim1, sep=""))
```

Along the same lines as described previously (in part two documentation): calculate “reverse-engineered” PIFs and “standardized mort”, and get summary stats.

```
#RE.PAFs.info<-merged[,c(unlist(strata.combos[j]), "outcome"), with=FALSE]
RE.PAFs<-cbind(merged[,c("outcome", "riskfactor"), with=FALSE],
              merged[,allmort.starting.point:allmort.ending.point]/merged[,totalmort.starting.p

setorder(RE.PAFs, riskfactor, outcome)
sum.stats.attr.mort.PAF<-sum.stats.maker(strata.sims.allmort)
RE.PAFs.sum.stats<-sum.stats.maker.RE.PAFs(RE.PAFs)

standardized.mort<-cbind(merged[,c("outcome", "riskfactor"), with=FALSE],
                        merged[,allmort.starting.point:allmort.ending.point]/merged[,pop.starti
setorder(standardized.mort, riskfactor, outcome)

standardized.mort.sum.stats<-sum.stats.maker(standardized.mort)
names(standardized.mort.sum.stats)[(length(names(standardized.mort.sum.stats))-4):length(names(s
paste("mort.per.100k.", names(standardized.mort.sum.stats)[(length(names(standardized.mort.sum.
```

Merge all the summary output we created (attributable mort, RE.PIF, standardized mort) into one super output file which we will call “strata.summary.stats”.

```
#merged.results<-merge(sum.stats.attr.mort.PAF, RE.PAFs.sum.stats)
merged.results<-reduce(list(sum.stats.attr.mort.PAF, RE.PAFs.sum.stats, standardized.mort.sum.st

strata.summary.stats<-merged.results
```

Finally, return all output of interest as a list of five elements: 1. summary stats for all three by specified strata of interest, 2. simulated values for attributable mort/incidence by specified strata of interest, 3. simulated values total mortality/incidence by specified strata of interest, 4. “Reverse-engineered” PIFs by specified strata of interest, 5. standardized attributable mortality by specified strata of interest,

```
return(list(strata.summary.stats, strata.sims.allmort, strata.sims.totalmort, RE.PAFs, standardi
})
```

Below is an obsolete function. It used to be used to calculate “overall” results (as in, not stratified by age/sex/race). What “overall” means became more complicated as summaries by different disease types became a result of interest, so this was scrapped in favor of making Sum.by.strata more flexible (work for disease types too, not just age/sex/race). But you can see the code is essentially the same, just without summing by strata.

```

Sum.by.overall<-function(allmort, totalmort, pop)
{
  allmort.dt<-as.data.table(allmort)
  totalmort.dt<-as.data.table(totalmort)
  pop.dt<-as.data.table(pop)

  cols.to.sum.allmort<-paste("V", 1:n.sims, sep="")
  cols.to.sum.totalmort<-paste("X", 1:n.sims, sep="")
  cols.to.sum.pop<-paste("Y", 1:n.sims, sep="")

  strata.sims.allmort<-allmort.dt[,lapply(.SD, sum), by=c("outcome", "disease_type", "riskfactor")]
  strata.sims.totalmort<-totalmort.dt[,lapply(.SD, sum), by=c("outcome", "disease_type"), .SDcols=
  strata.sims.pop<-pop.dt[,lapply(.SD, sum), .SDcols=cols.to.sum.pop]

  #merged<-merge(strata.sims.allmort, strata.sims.totalmort)
  #merged<-reduce(list(strata.sims.allmort, strata.sims.totalmort, strata.sims.pop), merge)
  #merged<-merge(merge(strata.sims.allmort, strata.sims.totalmort), strata.sims.pop, by="")
  merged<-merge(strata.sims.allmort, strata.sims.totalmort)
  #replicate to get n rows of same numbers
  strata.sims.pop<-map_dfr(seq_len(dim(merged)[1]), ~strata.sims.pop)
  merged<-cbind(merged, strata.sims.pop)

  allmort.starting.point<-which(names(merged)=="V1")
  allmort.ending.point<-which(names(merged)==paste("V", nsim1, sep=""))
  totalmort.starting.point<-which(names(merged)=="X1")
  totalmort.ending.point<-which(names(merged)==paste("X", nsim1, sep=""))
  pop.starting.point<-which(names(merged)=="Y1")
  pop.ending.point<-which(names(merged)==paste("Y", nsim1, sep=""))

  #RE.PAFs.info<-merged[,c(unlist(strata.combos[j]), "outcome"), with=FALSE]
  RE.PAFs<-cbind(merged[,c("outcome", "riskfactor"), with=FALSE],
                merged[,allmort.starting.point:allmort.ending.point]/merged[,totalmort.starting.p

  setorder(RE.PAFs, riskfactor, outcome)
  sum.stats.attr.mort.PAF<-sum.stats.maker(strata.sims.allmort)
  RE.PAFs.sum.stats<-sum.stats.maker.RE.PAFs(RE.PAFs)

  standardized.mort<-cbind(merged[,c("outcome", "riskfactor"), with=FALSE],
                          merged[,allmort.starting.point:allmort.ending.point]/merged[,pop.starti
  setorder(standardized.mort, riskfactor, outcome)

  standardized.mort.sum.stats<-sum.stats.maker(standardized.mort)
  names(standardized.mort.sum.stats)[(length(names(standardized.mort.sum.stats))-4):length(names(s
    paste("mort.per.100k.", names(standardized.mort.sum.stats)[(length(names(standardized.mort.sum.

  #merged.results<-merge(sum.stats.attr.mort.PAF, RE.PAFs.sum.stats)
  merged.results<-reduce(list(sum.stats.attr.mort.PAF, RE.PAFs.sum.stats, standardized.mort.sum.st

  strata.summary.stats<-merged.results

```

```
return(list(strata.summary.stats, strata.sims.allmort, strata.sims.totalmort, RE.PAFs, standardi
})
```

function to use to get summary stats by strata for joint PIFs (that is, for combination of shifts of multiple dietary factors). It is now identical to the Sum.by.strata function. (It used to be different, before sum.by.strata was made to be more flexible). Easier to change the function than change every instance of Sum.by.strata.joint being used in part three.

```

Sum.by.strata.joint<-function(allmort, totalmort, pop ,covar)
{
  allmort.dt<-as.data.table(allmort)
  totalmort.dt<-as.data.table(totalmort)
  pop.dt<-as.data.table(pop)

  strata<-covar

  cols.to.sum.allmort<-paste("V", 1:n.sims, sep="")
  cols.to.sum.totalmort<-paste("X", 1:n.sims, sep="")
  cols.to.sum.pop<-paste("Y", 1:n.sims, sep="")

  #strata.sims.allmort<-allmort.dt[,lapply(.SD, sum), by=c(strata, "outcome", "outcome_all", "dise
  #strata.sims.totalmort<-totalmort.dt[,lapply(.SD, sum), by=c(strata, "outcome", "outcome_all", "

  strata.sims.allmort<-allmort.dt[,lapply(.SD, sum), by=c(strata), .SDcols=cols.to.sum.allmort]
  strata.sims.totalmort<-totalmort.dt[,lapply(.SD, sum), by=c(strata[strata %in% c("age", "female"
  .SDcols=cols.to.sum.totalmort]
  strata.sims.pop<-pop.dt[,lapply(.SD, sum), by=c(strata[strata %in% c("age", "female", "race")]),

  #merged<-merge(strata.sims.allmort, strata.sims.totalmort)
  if(length(strata[strata %in% c("age", "female", "race")])>0 & dim(strata.sims.totalmort)[1]>1)
    merged<-merge(merge(strata.sims.allmort, strata.sims.totalmort), strata.sims.pop, by=strata[st
  if(length(strata[strata %in% c("age", "female", "race")])>0 & dim(strata.sims.totalmort)[1]==1)
  {
    merge.a<-cbind(strata.sims.allmort, strata.sims.totalmort)
    merged<-merge(merge.a, strata.sims.pop, by=strata[strata %in% c("age", "female", "race")])
  }
  if(length(strata[strata %in% c("age", "female", "race")])==0 & dim(strata.sims.totalmort)[1]>1)
  {
    merged<-cbind(merge(strata.sims.allmort, strata.sims.totalmort), strata.sims.pop)
  }
  if(length(strata[strata %in% c("age", "female", "race")])==0 & dim(strata.sims.totalmort)[1]==1)
    merged<-cbind(strata.sims.allmort, strata.sims.totalmort, strata.sims.pop)

  allmort.starting.point<-which(names(merged)=="V1")
  allmort.ending.point<-which(names(merged)==paste("V", nsim1, sep=""))
  totalmort.starting.point<-which(names(merged)=="X1")
  totalmort.ending.point<-which(names(merged)==paste("X", nsim1, sep=""))
  pop.starting.point<-which(names(merged)=="Y1")
  pop.ending.point<-which(names(merged)==paste("Y", nsim1, sep=""))

  #RE.PAFs.info<-merged[,c(unlist(strata.combos[j]), "outcome"), with=FALSE]
  RE.PAFs<-cbind(merged[,c(strata), with=FALSE],
    merged[,allmort.starting.point:allmort.ending.point]/merged[,totalmort.starting.p

  #setorder(RE.PAFs, outcome)
  setorderv(RE.PAFs, cols=c(strata))

```

```
sum.stats.attr.mort.PAF<-sum.stats maker(strata.sims.allmort)
RE.PAFs.sum.stats<-sum.stats maker.RE.PAFs(RE.PAFs)

standardized.mort<-cbind(merged[,c(strata), with=FALSE],
                        merged[,allmort.starting.point:allmort.ending.point]/merged[,pop.starti

setorderv(standardized.mort, cols=c(strata))
standardized.mort.sum.stats<-sum.stats maker(standardized.mort)
names(standardized.mort.sum.stats)[(length(names(standardized.mort.sum.stats))-4):length(names(s
  paste("mort.per.100k.",names(standardized.mort.sum.stats)[(length(names(standardized.mort.sum.

#merged.results<-merge(sum.stats.attr.mort.PAF, RE.PAFs.sum.stats)
merged.results<-reduce(list(sum.stats.attr.mort.PAF, RE.PAFs.sum.stats, standardized.mort.sum.st

strata.summary.stats<-merged.results

#return(list(strata.summary.stats, strata.sims.allmort, RE.PAFs))
return(list(strata.summary.stats, strata.sims.allmort, strata.sims.totalmort, RE.PAFs, standardi
}
```

Made obsolete by new version of Sum.by.strata. No longer in use.

```
Sum.by.overall.joint<-function(allmort, totalmort)
{
  allmort.dt<-as.data.table(allmort)
  totalmort.dt<-as.data.table(totalmort)

  cols.to.sum.allmort<-paste("V", 1:n.sims, sep="")
  cols.to.sum.totalmort<-paste("X", 1:n.sims, sep="")

  strata.sims.allmort<-allmort.dt[,lapply(.SD, sum), by=c("outcome", "outcome_all", "disease_type")]
  strata.sims.totalmort<-totalmort.dt[,lapply(.SD, sum), by=c("outcome", "outcome_all", "disease_t

  merged<-merge(strata.sims.allmort, strata.sims.totalmort)

  allmort.starting.point<-which(names(merged)==paste("V1", nsim1, sep=""))
  allmort.ending.point<-which(names(merged)==paste("V", nsim1, sep=""))
  totalmort.starting.point<-which(names(merged)==paste("X1", nsim1, sep=""))
  totalmort.ending.point<-which(names(merged)==paste("X", nsim1, sep=""))

  #RE.PAFs.info<-merged[,c(unlist(strata.combos[j]), "outcome"), with=FALSE]
  RE.PAFs<-cbind(merged[,c("outcome"), with=FALSE],
    merged[,allmort.starting.point:allmort.ending.point]/merged[,totalmort.starting.p

  setorder(RE.PAFs, outcome)

  sum.stats.attr.mort.PAF<-sum.stats.maker(strata.sims.allmort)
  RE.PAFs.sum.stats<-sum.stats.maker.RE.PAFs(RE.PAFs)

  merged.results<-merge(sum.stats.attr.mort.PAF, RE.PAFs.sum.stats, by="outcome", "outcome_all", "

  strata.summary.stats<-merged.results

  return(list(strata.summary.stats, strata.sims.allmort, RE.PAFs))
}
```